associations that are needed further up the stack (in the service or the view layer) must be loaded before the Hibernate `Session` is closed.

Unfortunately, there are bad ways to deal with the `LazyInitializationException` too. One quick fix would be to change the association in question to `FetchType.EAGER`. While this would work for the current business use case, the `FetchType.EAGER` is going to affect all other queries where the root entity of this association is fetched.

The fetching strategy is a query time responsibility, and each query should only fetch just as much data that is needed by the current business use case. On the other hand, `FetchType.EAGER` is a mapping time decision that is taken outside the business logic context where the association is meant to be used.

There is also the *Open Session in View* anti-pattern that is sometimes proposed as a solution for the `LazyInitializationException`.

14.3.3.2.4 The Open Session in View Anti-Pattern

Open Session in View is an architectural pattern that proposes to hold the Persistence Context open throughout the whole web request. This way, if the service layer fetched an entity without fully initializing all its associations further needed by the UI, then the view layer could silently trigger a Proxy initialization on demand.

Spring framework comes with a `javax.servlet.Filter`[1] implementation of the Open Session in View pattern. The `OpenSessionInViewFilter` gets a `Session` from the underlying `SessionFactory` and registers it in a `ThreadLocal` storage where the `HibernateTransactionManager` can also locate it. This service layer is still responsible for managing the actual JDBC or JTA transaction, but the `Session` is no longer closed by the `HibernateTransactionManager`[2].

[1] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/orm/hibernate5/support/OpenSessionInViewFilter.html

[2] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/orm/hibernate5/HibernateTransactionManager.html

To visualize the whole process, consider the following sequence diagram:

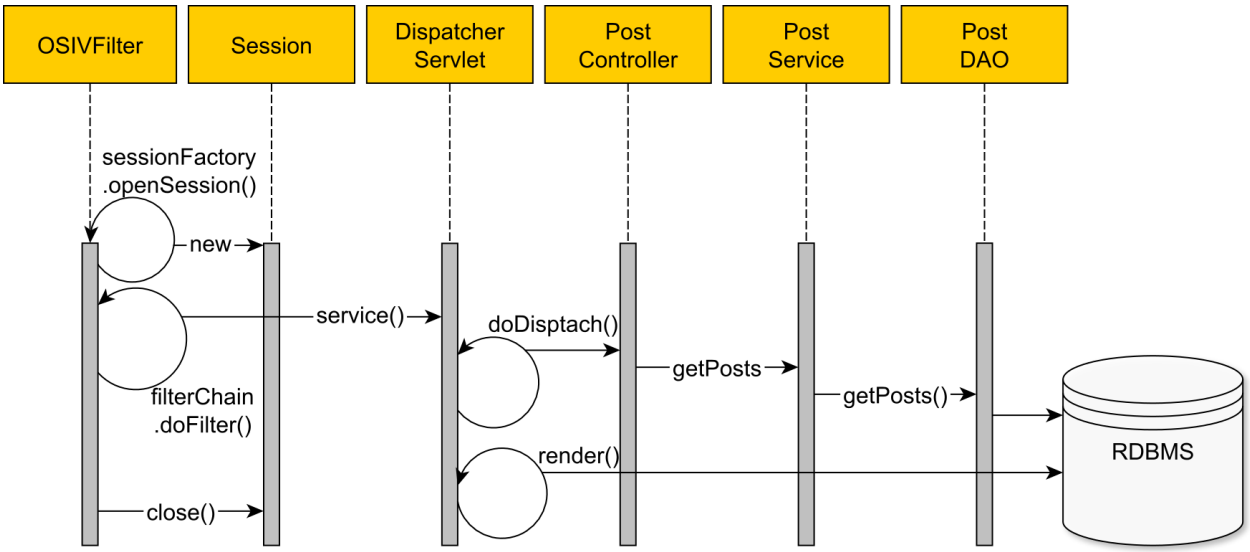

Figure 14.3: Open Session in View lifecycle

- The `OpenSessionInViewFilter` calls the `openSession` method of the underlying `SessionFactory` and obtains a new `Session`.
- The `Session` is bound to the `TransactionSynchronizationManager`[3].
- The `OpenSessionInViewFilter` calls the `doFilter` of the `javax.servlet.FilterChain` object reference and the request is further processed
- The `DispatcherServlet`[4] is called, and it routes the HTTP request to the underlying `PostController`.
- The `PostController` calls the `PostService` to get a list of `Post` entities.
- The `PostService` opens a new transaction, and the `HibernateTransactionManager` reuses the same `Session` that was opened by the `OpenSessionInViewFilter`.
- The `PostDAO` fetches the list of `Post` entities without initializing any lazy association.
- The `PostService` commits the underlying transaction, but the `Session` is not closed because it was opened externally.
- The `DispatcherServlet` starts rendering the UI, which, in turn, navigates the lazy associations and triggers their initialization.
- The `OpenSessionInViewFilter` can close the `Session`, and the underlying database connection is released as well.

At a first glance, this might not look like a terrible thing to do, but, once you view it from a database perspective, a series of flaws start to become more obvious.

[3] http://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/support/TransactionSynchronizationManager.html

[4] http://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/web/servlet/DispatcherServlet.html

The service layer opens and closes a database transaction, but afterward, there is no explicit transaction going on. For this reason, every additional statement issued from the UI rendering phase is executed in auto-commit mode. Auto-commit puts pressure on the database server because each statement must flush the transaction log to disk, therefore causing a lot of I/O traffic on the database side. One optimization would be to mark the `Connection` as read-only which would allow the database server to avoid writing to the transaction log.

There is no separation of concerns anymore because statements are generated both by the service layer and by the UI rendering process. Writing integration tests that assert the number of statements being generated requires going through all layers (web, service, DAO), while having the application deployed on a web container. Even when using an in-memory database (e.g. HSQLDB) and a lightweight web server (e.g. Jetty), these integration tests are going to be slower to execute than if layers were separated and the back-end integration tests used the database, while the front-end integration tests were mocking the service layer altogether.

The UI layer is limited to navigating associations which can, in turn, trigger N+1 query problems, as previously explained. Although Hibernate offers `@BatchSize`[5] for fetching associations in batches, and `FetchMode.SUBSELECT`[6] to cope with this scenario, the annotations are affecting the default fetch plan, so they get applied to every business use case. For this reason, a data access layer query is much more suitable because it can be tailored for the current use case data fetch requirements.

Last but not least, the database connection is held throughout the UI rendering phase which increases connection lease time and limits the overall transaction throughput due to congestion on the database connection pool. The more the connection is held, the more other concurrent requests are going to wait to get a connection from the pool.

The Open Session in View is a solution to a problem that should not exist in the first place, and the most likely root cause is relying exclusively on entity fetching.

If the UI layer only needs a view of the underlying data, then the data access layer is going to perform much better with a DTO projection. A DTO projection forces the application developer to fetch just the required data set and is not susceptible to `LazyInitializationException(s)`.

This way, the separation of concerns is no longer compromised, and performance optimizations can be applied at the data access layer since all statements are confined to the boundaries of the currently executing transaction.

[5] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/annotations/BatchSize.html
[6] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/annotations/FetchMode.html#SUBSELECT

14.3.3.2.5 Temporary Session Lazy Loading Anti-Pattern

Analogous to the Open Session in View, Hibernate offers the `hibernate.enable_lazy_load_no_trans` configuration property which allows an uninitialized lazy association to be loaded outside of the context of its original Persistence Context.

```
<property name="hibernate.enable_lazy_load_no_trans" value="true"/>
```

With this configuration property in place, the following code snippets can be executed without throwing any `LazyInitializationException`:

```
List<PostComment> comments = null;

EntityManager entityManager = null;
EntityTransaction transaction = null;
try {
    entityManager = entityManagerFactory().createEntityManager();
    transaction = entityManager.getTransaction();
    transaction.begin();

    comments = entityManager.createQuery(
        "select pc " +
        "from PostComment pc " +
        "where pc.review = :review", PostComment.class)
    .setParameter("review", review)
    .getResultList();

    transaction.commit();
} catch (Throwable e) {
    if ( transaction != null && transaction.isActive())
        transaction.rollback();
    throw e;
} finally {
    if (entityManager != null) {
        entityManager.close();
    }
}
for(PostComment comment : comments) {
    LOGGER.info("The post title is '{}'", comment.getPost().getTitle());
}
```

Behind the scenes, a temporary `Session` is opened just for initializing every `post` association. Every temporary `Session` implies acquiring a new database connection, as well as a new database transaction.

The more associations being loaded lazily, the more additional connections are going to be requested which puts pressure on the underlying connection pool. Each association being loaded in a new transaction, the transaction log is forced to flush after each association initialization.

Just like Open Session in View, the `hibernate.enable_lazy_load_no_trans` configuration property is an anti-pattern as well because it only treats the symptoms and does not solve the actual cause of the `LazyInitializationException`.

By properly initializing all lazy associations prior to closing the initial Persistence Context, and switching to DTO projections where entities are not even necessary, the `LazyInitializationException` is prevented in a much more efficient way.

14.3.3.3 Associations and pagination

As previously explained, paginating result sets has many benefits, from lowering the response time to ensuring that the application works with the ever increasing data sets. Also, fetching a collection with the `join fetch` JPQL directive can prevent N+1 query problems and `LazyInitializationException(s)` as well. Unfortunately, mixing collection fetching and pagination does not work very well together.

Collections must always be fetched fully because otherwise the collection size might not be consistent with the number of child entries associated with a given parent. On the other hand, SQL pagination can truncate the collection before returning all child records, therefore breaking the aforementioned consistency guarantee.

To visualize this process, the following entity query is going to load a list of `Post` entities, filtered by their `title`, and also, fetch all comments associated with a given `Post` record.

When specifying a `maxResults` restriction:

```
List<Post> posts = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "left join fetch p.comments " +
    "where p.title like :title " +
    "order by p.id", Post.class)
.setParameter("title", titlePattern)
.setMaxResults(50)
.getResultList();
```